qss

Tables, one vocabulary, presentations and contracts for Emacs

A short manual

Friedrich Dominicus

Version 0.12.1

Contents

1 Introduction **4**
- 1.1 Installation . . . 4
- 1.2 A first session . . . 4

2 Tables and rows **5**
- 2.1 A table is a value . . . 5
- 2.2 The protocol of rows . . . 5
- 2.3 Columns . . . 5

3 The vocabulary **6**
- 3.1 Sources and words . . . 6
- 3.2 Groups and joins . . . 6
- 3.3 Conditions . . . 7

4 The REPL **8**
- 4.1 Input . . . 8
- 4.2 Keys . . . 8

5 Actions at point **9**

6 SQLite **10**
- 6.1 Queries . . . 10
- 6.2 Storing: `into` . . . 10
- 6.3 Changing rows . . . 11

7 Nushell **12**

8 Presentations and accept **13**
- 8.1 The idea . . . 13
- 8.2 Presentations . . . 13
- 8.3 Accept . . . 14
- 8.4 Types . . . 14
- 8.5 Examples . . . 14
- 8.6 Commands with typed arguments . . . 15
- 8.7 Compared with CLIM . . . 15

9 Contracts **16**
- 9.1 Why . . . 16
- 9.2 Blame . . . 16
- 9.3 The interface . . . 16
- 9.4 The language . . . 17
- 9.5 For your own code . . . 17

10 A row type of your own **18**
10.1 The row object and the source . . . 18
10.2 The protocol . . . 18
10.3 A library is a file . . . 19
10.4 Storing and a command of its own . . . 20
10.5 A contract . . . 20

11 Reference **21**
11.1 Words . . . 21
11.2 Sources . . . 21
11.3 Options . . . 22
11.4 Faces . . . 23
11.5 Presentation types . . . 23
11.6 Commands with typed arguments . . . 23
11.7 Named contracts . . . 24
11.8 Contracts . . . 24

12 Where the ideas come from **27**

A Files and tests **28**

1 Introduction

qss brings the idea of Nushell to Emacs: data as tables, and one small vocabulary that works on all of them — files, processes, lists of property lists, rows of an SQLite table, and whatever Nushell itself produces. Around this idea are a few others, older ones:

- on SQLite the words become SQL, as in dbplyr, and SQLite is both source and store;
- what is shown stays the object it shows and can be pointed at later, as in CLIM: *presentations* and *accept*;
- the public interface is checked by *contracts* with blame, as in Racket.

Everything is Emacs Lisp and runs inside Emacs (29.1 or later). Only what is built into Emacs is used: `vtable`, `sqlite`, `cl-generic` and advice.

This manual explains the ideas and how they fit together. The README of the package is the short version; the docstrings (`C-h f`) are the detailed one. The reference in chapter 11 is written from the code itself.

1.1 Installation

```
(package-vc-install "https://github.com/FDominicus/qss")
(require 'qss)
```

Or put the directory on `load-path` and `(require 'qss)`. Nushell is optional; without it everything else works.

1.2 A first session

`M-x qss` opens the REPL. Type a pipe, press `RET`, and the table appears right below:

```
~/ qss> (files "~") (where (> :size "10mb")) (sort-by size :desc) (first 5)
5 rows  [buffer]  files ~/ | where (> :size "10mb") | sort-by size desc | first 5
name                type  size modified         mode       user
lisp.virtualmem     file   11M 2026-06-01 14:35 -rw-rw-r-- frido
...
~/ qss> (select name size)
```

The second line starts with a word, so it continues the table above. The title line repeats every stage of the pipe, so you always see how a table came about. `RET` on a row opens the file, `a` offers what can be done with it, and `(sqlite (qss-demo-database) "Orders")` gives a small database to try the SQL part with.

2 Tables and rows

2.1 A table is a value

A table (`qss-table`) is a title, a list of columns, a list of rows and, optionally, a way to build it again. It is an ordinary Lisp value: it can be stored in a variable, passed on, shown.

The rows are *not* turned into strings or records of strings. A row of `(files)` is a `qss-file`, a row of `(processes)` a `qss-process`, a row from SQLite a property list. The table only knows how to ask a row for its fields. That is why a filtered file still opens with `RET`, and why the same words work on all of them.

2.2 The protocol of rows

What a row is, is said by a few generic functions, with a method per type of row:

`qss-columns`	the columns shown for a row
`qss-field`	the raw value of a field (filtering and sorting use it)
`qss-default-action`	what `RET` does
`qss-actions`	what `a` offers: a list of (*label* . *function*)
`qss-stored-fields`	fields stored by `into` besides the columns
`qss-presentation-type`	the type for `qss-accept` (chapter 8)
`qss-presentation-label`	a short text for it
`qss-file-name`	the file a row stands for, if any

Property lists and alists have methods already, so any list of property lists is a table:

```
(qss-> '((:name "Ada" :born 1815) (:name "Grace" :born 1906))
       (where (< :born 1900))
       collect)
;; => ((:name "Ada" :born 1815))
```

2.3 Columns

A column is a field name, or (*name* . *plist*) with `:type` (`number`, `size`, `duration`, `time`), `:format`, `:align` and `:width`. The type decides display, alignment and sorting: a `size` is bytes shown as `9.8k`, a `duration` nanoseconds shown as `3hr 20min`, a `time` a Lisp time shown as `2026-10-22 14:40`.

3 The vocabulary

3.1 Sources and words

A pipe begins with a source and goes on with words:

```
(qss-> (sqlite (qss-demo-database) "Orders")
       (where (> :amount 100))
       (sort-by order-date :desc)
       (select order-no amount)
       collect)
```

In the REPL and in `M-x qss-eval` the same is written without `qss->`. The sources are `files`, `processes`, `run`, `sqlite`, `tables`, `nu` and `contracts`; any list is one too. The words:

`(where` *cond*`)`	rows for which the condition holds (`filter` is the same)
`(select` *col…*`)`, `(reject` *col…*`)`	keep, drop columns
`(sort-by` *col* `[:desc])`	sort; missing values last
`(first [`*n*`])`, `(last [`*n*`])`	the first, last rows
`(get` *col*`)`	the values of one column
`(group-by` *col… agg…*`)`	one row per group: keys, count, aggregates; `RET` shows the group
`(summarize` *agg…*`)`	one row: count and aggregates over all rows
`(join` *table* `:on` *col*`)`	rows of both tables whose columns match
`(update-rows :`*col v…*`)`, `(delete-rows)`	change, delete stored rows (section 6.3)
`(each` *form*`)`	*form* with `it` bound to each row; the results are the new rows
`(nu` *command*`)`	the table through Nushell (section 7)
`(to` *format*`)`	the table as text: `text`, `org`, `md`, `csv`, `tsv`
`(into sqlite` *name …*`)`	store the table (section 6.2)
`columns`, `count`, `collect`, `sql`	column names, number of rows, the rows as a list, the SQL

Every word returns a table again (or a query, on SQLite), except `count`, `collect`, `to` and `sql`. A derived table knows its way back to the source: `g` in its buffer reads the source again and applies the words once more.

3.2 Groups and joins

```
qss> (sqlite (qss-demo-database) "Orders") (group-by status (sum amount) (max order-date))
status     count  sum-amount  max-order-date
cancelled      1       150.0  2026-10-12
```

```
paid           3       245.5  2026-10-02
               2     1299.99  2026-10-05
```

`group-by` takes the columns to group by and any number of aggregates: `(sum col)`, `(avg col)`, `(min col)`, `(max col)` and `(count col)`, the number of values present. They are named `sum-amount` and so on, or as told: `(sum amount :as total)`. `count`, the number of rows, is always there. As in SQL, missing values are left out of the aggregates. The groups are sorted by their keys, missing values last, in memory and on SQLite alike; a sum or average of sizes is a size again. `RET` on a group shows its rows. `summarize` is the same without groups: one row over all rows (also over none: then the count is 0).

`join` puts the rows of two tables side by side where their columns match:

```
qss> (sqlite (qss-demo-database) "Orders")
       (join (sqlite (qss-demo-database) "Customers") :on customer-no)
       (group-by city (sum amount)) (sort-by sum-amount :desc)
```

The other table is written as a source, like the start of a pipe. `:on` names the column (the same on both sides), a pair `(left right)` for different names, or a list of pairs; without `:on`, all columns both tables have. `:kind left` keeps the rows of the left table without a partner. A column both sides have appears once if it is joined on; otherwise the right one gets the suffix `-2`. Missing values never match, as in SQL. The rows of a join are data (property lists). The options are data like column names: in a pipe they are not evaluated.

3.3 Conditions

Columns are keywords; everything else is Lisp, variables included:

```
(where (> :size 10000))
(where (and (>= :date since) (in :kind kinds)))
(where (like :name "%.el"))
(where (is-null :invoice-no))
```

The comparisons mean what they mean in SQL. `=`, `<` … work on numbers, strings and times (`(> :modified "2026-10-20")` works); sizes and durations can be written as in Nushell (`(> :size "10kb")`, `(< :took "1hr")`); `like` ignores case; and a missing value is NULL: comparing with it is *unknown*, neither true nor false, and `where` keeps only what is true. So `(not (= :city "Bruchsal"))` does not keep rows without a city — in memory exactly as in SQLite.

If the symbol `it` occurs, the condition is ordinary Lisp with `it` bound to the row object:

```
(where (string-prefix-p "." (qss-field it 'name)))
```

4 The REPL

4.1 Input

- Several forms, or one form beginning with a source, are a pipe.
- A form beginning with a word continues the last table, like `$in` in Nushell; so does a line beginning with `|`.
- Anything else is Lisp. The last table is in the variable `qss-last`.
- `!`*`program args`* runs a program directly, without a shell; its output is shown and is also the last table (one row per line, in the column `line`).
- `cd` *`dir`* changes the directory; the prompt shows it, and `files`, `nu`, `run` and `!` run there — on the remote host in a TRAMP directory.

```
~/ qss> cd ~/src/project
~/src/project/ qss> !git log --oneline -20
~/src/project/ qss> (where (like :line "%fix%"))
~/src/project/ qss> | (nu "length")
```

4.2 Keys

`RET`	on the input: run it; on a row: its default action (a directory: `cd` there)
`a`, right mouse button	the actions at point (chapter 5)
`w`	copy the whole table as text; `C-u w` asks for the format
`S`	sort by the column at point
`e`, `d`, `u`	in a table buffer of an SQLite table: change a cell, mark a row for deletion, undo (section 6.3)
`C-c C-v`	take a value from a table into the input, by a click
`C-c C-f`	take the name of a file into the input
`M-p`, `M-n`	history
`TAB`	complete words, sources, column names, file names
`C-c M-o`	clear the buffer
`[buffer]`	the table in a buffer of its own, where `g` reads it again

Long tables show the first `qss-repl-max-rows` rows and a `[show all]` button.

5 Actions at point

`a`, or the right mouse button, on a cell offers four groups:

1. the actions of the row object — open the file, Dired, kill the process … (`qss-actions`);
2. words with the value under point: `(where (= :status "paid"))`, `(where (/= …))`, `(where (> :amount 120.0))` for numbers, `(where (is-null :city))` for an empty cell, the day for a time; then `sort-by`, `group-by`, `select` and `reject` on that column, and copying the value;
3. commands with typed arguments whose first argument fits the row or the value (section 8.6);
4. copying the whole table as text, Org, Markdown, CSV or TSV.

A word continues the *whole* table, not only the rows the REPL shows, and on SQLite it continues the query, so the condition becomes SQL. In the REPL the word is submitted as the next input — visible and in the history — and it continues the table that was clicked, even an older one; what you had typed at the prompt stays. In a table buffer the result opens in a buffer of its own.

`(to org)` and `w` show the cells as the table does (`9.8k`); `(to csv)` and `(to tsv)` carry the plain values for other programs: sizes in bytes, durations in nanoseconds, times with seconds.

6 SQLite

6.1 Queries

On an SQLite source the pipe stays a query until the rows are needed. `where`, `sort-by`, `first`, `count`, `select`, `reject`, `columns`, `group-by`, `summarize` and `join` (with a query of the same file) become SQL with `?` parameters; a word that cannot (a condition with `it`, `each`, `last`, `get`) fetches the rows and goes on in memory. After `group-by`, a `where` on the aggregates is SQL too (what SQL calls HAVING), and `RET` on a group fetches its rows with a query of its own.

```
qss> (sqlite (qss-demo-database) "Orders") (where (> :amount 100))
       (sort-by order-date :desc) (first 20) sql
SELECT *
FROM "Orders"
WHERE ("Amount" > ?)
ORDER BY "OrderDate" IS NULL, "OrderDate" DESC
LIMIT ?
Parameters: (100 20)
```

Values never become text in the SQL. Column and table names become keys mechanically, never by translation: `OrderDate` → `order-date`, `HT_Orders` → `ht-orders`, `Straße` → `strasse`; the way back is looked up in the schema. `"10kb"` works in SQL too: for `<`, `>`, `<=`, `>=` on every column without a text type, for `=`, `/=`, `in` only on numeric columns — so that SQL returns the same rows as memory.

6.2 Storing: `into`

`into` is the way back. The table of a pipe goes into SQLite and comes out as a query, so the words go on there.

```
qss> (files "~") (where (> :size "10mb")) (into sqlite "big-files")
qss> (sqlite "big-files") (where (> :modified "2026-01-01"))
qss> (nu "sys disks") (into sqlite "disks" :append)
```

- Without `:file` the table goes into the central file `qss-database` (`qss.sqlite3` in `qss-data-directory`, by default `~/.emacs.d/`); `(sqlite "big-files")` and `(tables)` read from there.
- Keep the SQLite files on a local disk. If `~/.emacs.d` lies on a network drive that several Emacs use at once, SQLite's locking cannot be trusted there; give each Emacs its own `qss-data-directory`.
- An existing table is not touched unless `:replace` or `:append` says so. All in one transaction.

- Sizes and durations are stored as INTEGER (bytes, nanoseconds), times as ISO text `2026-10-22 07:11:05` (`qss-time-format`); the types of `qss` are recorded in a table `_qss_columns`, so sizes show as `9.8k` again.
- A file keeps its path, so `RET` on the stored row opens the file; other row types name the fields to keep with `qss-stored-fields`.

6.3 Changing rows

SQLite is a file to work in, not only to read. A table buffer of an SQLite table (`[buffer]` after a table in the REPL) can be edited:

`e`	change the cell at point; `C-u e` makes it empty (NULL)
`d`	mark the row for deletion (again: unmark)
`u`	forget the changes of the row
`C-c C-c`	write all changes, in one transaction
`C-c C-k`	forget all changes

The changes are shown where they are made — changed cells highlighted, deleted rows struck through, a count in the mode line — and written only by `C-c C-c`, all or nothing: if one row cannot be written (someone deleted it meanwhile), none is. `g` and killing the buffer ask while there are changes. A numeric column takes numbers, also `"10kb"` and `"3hr"`; empty input there means NULL.

For many rows at once there are two words; they return how many rows they changed:

```
qss> (sqlite "big-files") (where (> :size "1gb")) (update-rows :note "archive")
qss> (sqlite (qss-demo-database) "Orders") (where (= :status "cancelled")) (delete-rows)
```

Only the rows of the query change: its `where`, `first` and `sort-by` count. Without `where` or `first` the words refuse to touch every row, unless `:all-rows t` says so. The values are Lisp and evaluated.

Rows are found by their rowid. So only queries straight on a table can be changed — not views, joins or groups; `qss` says so instead of guessing.

7 Nushell

`(nu command)` is a source: Nushell runs the command in `default-directory` and the result becomes a table. In the middle of a pipe `nu` works the other way round as well: the table so far goes to Nushell as `$in`, and what comes back is a table again (or a single value).

```
qss> (nu "ps | where cpu > 5")
qss> (nu "open export.csv") (where (> :amount 100)) (sort-by date :desc)
qss> (files) (nu "where size > 10kb | get size | math sum")
qss> (sqlite (qss-demo-database) "Orders") (where (is-null :status))
       (nu "to csv | save -f open-orders.csv")
```

Both ways use NUON, Nushell's own format, which `qss` reads and writes itself (`qss-nuon.el`). Unlike JSON it keeps the types: file sizes, durations and datetimes arrive as columns of the types `size`, `duration` and `time`, and go back to Nushell as such, so `where size > 10kb` works on a table from Emacs.

8 Presentations and accept

8.1 The idea

The idea comes from CLIM, the Common Lisp Interface Manager, and from Dynamic Windows on the Symbolics Lisp machines before it: *output remembers what it shows*, so it can serve as input later. In a CLIM listener you type a command, and where it wants a file you click a file that some earlier output printed; the program receives the file object, not its name as text.

`qss` does the same in Emacs buffers. A buffer is the store: whatever is shown in a visible window can be chosen.

8.2 Presentations

A *presentation* is a piece of text together with an object and a *presentation type*.

- Every **row** of a table is a presentation of its row object: a file row of its `qss-file` (type `file`, or `directory`), a process row of its process, a row of `tables` of a `db-table`, a row of data of a `row` — or of a `file`, if it has a `path`.
- Every **cell** is a presentation of its value: `size`, `duration` and `time` from the type of the column, otherwise from the value (`integer`, `number`, `string`, `time`, `symbol`). Empty cells are none.
- What the **REPL prints** — a number, a line of text, a time — is one too.
- **Any text in any buffer**, made with `qss-present`.

The text stays plain text; the object rides along in the text property `qss-presentation`. Here is a buffer of notes that presents some files; each name is as good as a file row in a table:

```
(with-current-buffer (get-buffer-create "*to read*")
  (erase-buffer)
  (insert "Files to look at:\n")
  (dolist (f (directory-files "~/notes" t "\\.org\\'"))
    (insert "  ")
    (qss-present (list :path f) 'file (file-name-nondirectory f))
    (insert "\n"))
  (display-buffer (current-buffer)))
```

8.3 Accept

`(qss-accept type prompt)` asks for an object of a type.

1. It looks at **every visible window** — not at hidden buffers: what you see is what you can choose — and collects the presentations of the type or of any type below it.
2. It **highlights** them; the prompt tells how many: "with file: (3 shown, click one or type)".
3. You answer by a **click** on one (or `RET` on it): the answer is the object itself. Or you **type** its label, with completion over the highlighted ones. Or you type something new, and the text is **parsed** for the type: a path for `file`, `"10kb"` for `size`, `"3hr"` for `duration`, `"2026-10-22"` for `time`.
4. `C-g` gives up. The highlighting is gone either way.

A row whose object fits is offered as a whole row; otherwise its cells are looked at. So `file` offers file rows, `value` the cells.

8.4 Types

```
t ─┬─ object ─┬─ file − directory
   │          ├─ process, db-table, row
   └─ value ──┬─ number − integer ─┬─ size
              │                    └─ duration
              └─ string, time, symbol
```

A directory is a file too, a size a number: accepting `file` offers directories as well, accepting `directory` only directories. New types come with `qss-define-presentation-type`, with a parser for typed text; a type without one uses the parser of the nearest type above it. Section 11.5 lists them all.

8.5 Examples

A value into the input. In the REPL, while typing, `C-c C-v` accepts a `value`:

```
qss> (sqlite (qss-demo-database) "Orders")
...
    4714           3 2026-10-02   45.0 paid
qss> (where (= :customer-no ▮        C-c C-v: all cells light up; click the 3
qss> (where (= :customer-no 3▮
```

A number goes in as a number, text as a string, a time as `"2026-10-22 07:11:05"`. `C-c C-f` does the same with the name of a file: `!wc -l C-c C-f`, click a file row.

Two files from two places. Show two directories, each in a buffer of its own (`[buffer]`), side by side. On `main.el` in the left window, `a` → *Compare with file…*: the left row is the first file; every file row in both windows lights up; click `main.el` on the right, and `diff` shows the differences.

Only what fits. On a file, *Copy into directory…* lights up only directories — in every visible window, the notes buffer included.

A value as the start. On a size cell, `a` offers *Files larger than this, here.*

```
(let ((dir (qss-accept 'directory "Back up into: ")))
  (copy-directory "~/notes" (qss-file-name dir)))
```

8.6 Commands with typed arguments

`qss-define-command` defines a command whose arguments are accepted by type, after `define-command` in CLIM:

```
(qss-define-command my-open-both "Open side by side"
    ((a file "Open: ") (b file "next to: "))
  "Show the files A and B side by side."
  (find-file (qss-file-name a))
  (find-file-other-window (qss-file-name b)))
```

- `M-x my-open-both` accepts both files, each by a click or by typing.
- On every presentation of the type of the first argument — every file row, every presented file in any buffer — `a` and the right mouse button offer *Open side by side….* That presentation is the first argument; the others are accepted.
- `qss-file-name` turns what was chosen into a file name: a file row, a stored row, a typed path, or a row of an own type with a method for it.

8.7 Compared with CLIM

qss takes the core — presentations with types, accept with highlighting, commands with typed arguments offered on the objects — and leaves much out: there are no presentation translators besides the first argument of a command, no parsing of whole command lines in an input editor, no views, and only visible windows are searched. What it adds is that every Emacs buffer can take part.

9 Contracts

9.1 Why

qss is built around generic functions that others — or you, later — fill with methods. Tests check examples; contracts check every call. A contract says what a function takes and what it returns, and when it is broken, the message says *who* broke it:

```
qss> (processes) (first -1)
Contract violation: qss-first: argument 2 must satisfy natnump, got -1
  blame: the caller (outside the checked functions)
  contract: (-> qss-tabular (opt natnump) qss-result)
```

```
Contract violation: qss-columns: the result must satisfy
(listof qss-column-spec), got (name 42) (element 1 is 42, not qss-column-spec)
  blame: qss-columns itself, the method for my-thing
  contract: (-> any (listof qss-column-spec))
```

9.2 Blame

The contracts follow Racket. A contract is an `:around` advice on the function: it sees the arguments before the function runs and the result before it returns, so it knows the direction of the fault exactly.

- An argument that breaks the contract is the fault of the **caller**. Every contract binds the name of its function while it runs, so if the caller is a checked function itself, it is named: "blame: the caller, qss-continue". The checked functions are the boundaries, as modules are in Racket.
- A result that breaks it is the fault of the **function** — for a generic function, of the method for the type of the first argument. This is where own row types are caught.

9.3 The interface

The whole interface is in one file, `qss-interface.el`, readable as the reference of qss; section 11.8 lists it. The protocol of rows comes first, since methods for own types are where mistakes happen:

```
(qss-define-contract qss-columns (-> any (listof qss-column-spec)))
(qss-define-contract qss-sqlite  (-> stringp (opt (or/c stringp symbolp))
                                     qss-relation-p))
(qss-define-contract qss-accept  (-> qss-presentation-type-name (opt stringp) any))
```

`C-h f` shows the contract of a function, and the contracts are a source: `(contracts) (where (like :function "%sql%"))`.

The contracts stand beside the code, not in it. They hold for every method of a generic function, also methods added later, and survive redefinition. They are put only on the boundaries: functions that run once per row or per cell (`qss-field`, the formats of values) have none. There they doubled the time of a pipe over 20000 rows; at the boundaries they cost a few percent. `qss-contract-enabled` turns them all off.

9.4 The language

`stringp`, `natnump`, `qss-table-p` …	a predicate
`any`	anything
`(or/c …)`, `(and/c …)`, `(not/c c)`	one of, all of, not
`(one-of v…)`	one of these values
`(listof c)`, `(list/c …)`, `(cons/c a d)`	lists and conses
`(lambda (x) …)`	a predicate written in place
`(-> arg… result)`	a function; `(opt c)` an optional argument (nil is always allowed), `(rest c)` the rest
a name	a contract named with `qss-define-contract-type`

A function passed as an argument is checked to be a function, but not wrapped.

9.5 For your own code

`qss-contract.el` depends on nothing else of `qss` and can check any Emacs Lisp:

```
(require 'qss-contract)
(qss-define-contract-type 'my-weight '(and/c numberp (lambda (w) (>= w 0))))
(qss-define-contract my-tare (-> my-weight my-weight my-weight))
```

10 A row type of your own

This chapter builds a complete row type: the Emacs Lisp libraries on `load-path`. It shows how the parts of qss fit together — a library becomes a table row, a presentation of a file, something to store in SQLite, the argument of a command, and a checked source. The code is the file `doc/qss-manual-example.el`; the tests check that it works.

10.1 The row object and the source

Any Lisp object can be a row. Here a structure, and a function that returns the table:

```
(cl-defstruct (my-library (:constructor my-library--make) (:copier nil))
  "An Emacs Lisp library: its NAME (without .el), its PATH and SIZE."
  name path size)
```

```
(defun my-libraries (&optional regexp)
  "Return a table of the Emacs Lisp libraries on `load-path'.
With REGEXP, only those whose name matches it."
  (let ((seen (make-hash-table :test #'equal)) (rows nil))
    (dolist (dir load-path)
      (when (file-directory-p dir)
        (dolist (file (directory-files dir t "\\.el\\(?:\\.gz\\)?\\'" t))
          (let ((name (replace-regexp-in-string "\\.el\\(?:\\.gz\\)?\\'" ""
                                                (file-name-nondirectory file))))
            (when (and (not (gethash name seen))
                       (or (null regexp) (string-match-p regexp name)))
              (puthash name t seen)
              (push (my-library--make
                     :name name :path file
                     :size (file-attribute-size (file-attributes file)))
                    rows))))))
    (qss-make-table (if regexp (format "libraries %s" regexp) "libraries")
                    (nreverse rows) nil
                    (lambda () (my-libraries regexp)))))

;; A short name for pipes: (libraries "^subr")
(setf (alist-get 'libraries qss-sources) 'my-libraries)
```

10.2 The protocol

Three methods make a library a row: its columns, its fields, what `RET` and `a` do. Fields need not be columns: `compiled` can be used in conditions but is not shown.

```
(cl-defmethod qss-columns ((_ my-library))
  "Return the columns of a library row."
  '((name :width 30) (size :type size) path))

(cl-defmethod qss-field ((row my-library) name)
  "Return field NAME of the library ROW."
  (pcase name
    ('name (my-library-name row))
    ('size (my-library-size row))
    ('path (my-library-path row))
    ('compiled (let ((el (string-remove-suffix ".gz" (my-library-path row))))
                 (file-exists-p (concat (file-name-sans-extension el) ".elc"))))))

(cl-defmethod qss-default-action ((row my-library))
  "Visit the library ROW."
  (find-file (my-library-path row)))

(cl-defmethod qss-actions ((row my-library))
  "Return the actions of the library ROW."
  (ignore row)
  (list (cons "Visit" #'qss-default-action)
        (cons "Load" (lambda (r)
                       (load (my-library-path r))
                       (message "Loaded %s" (my-library-name r))))
        (cons "Describe" #'qss-describe)))
```

Now this works:

```
qss> (libraries "^qss-") (sort-by size :desc) (first 5)
qss> (libraries) (where (> :size "100kb")) count
qss> (libraries "^subr") (where (not (qss-field it 'compiled)))
```

The last line uses the field `compiled`; because it calls a function, it is a condition in Lisp with `it`, not one that could become SQL.

10.3 A library is a file

A new presentation type below `file`, a method that gives it to library rows, a label, and a method for `qss-file-name`:

```
(defun my-library-named (text)
  "Return the library called TEXT, or nil."
  (car (qss-table-rows
        (my-libraries (concat "\\`" (regexp-quote text) "\\'")))))

(qss-define-presentation-type 'library 'file #'my-library-named
  "An Emacs Lisp library on `load-path'; its name can be typed.")

(cl-defmethod qss-presentation-type ((_ my-library))
  "A library row is a `library', and so a `file'."
  'library)
```

```
(cl-defmethod qss-presentation-label ((row my-library))
  "Return the name of the library ROW."
  (my-library-name row))

(cl-defmethod qss-file-name ((row my-library))
  "Return the file of the library ROW, for the commands on files."
  (my-library-path row))
```

With that, everything that takes a `file` takes a library: *Compare with file…* and *Copy into directory…* appear on library rows, `C-c C-f` inserts their path, and `(qss-accept 'library)` reads a typed name such as `subr`.

10.4 Storing and a command of its own

The path is a column already, so nothing more needs to be stored; a stored row has a `path` and is a file again. A command for libraries only:

```
(cl-defmethod qss-stored-fields ((_ my-library))
  "A stored library keeps its path (it is a column already) and nothing more."
  nil)
```

```
(qss-define-command my-library-requires "Show what it requires"
    ((library library "Library: "))
  "Show the features LIBRARY requires, as a table."
  (qss-display
   (qss-make-table (format "requires of %s" (qss-presentation-label library))
                   (with-temp-buffer
                     (insert-file-contents (qss-file-name library))
                     (let (found)
                       (while (re-search-forward "^(require '\\([^) \n]+\\)" nil t)
                         (push (list :feature (match-string 1)) found))
                       (nreverse found)))
                   '(feature))))
```

On a library row, `a` now offers *Show what it requires*, and `M-x my-library-requires` accepts a library by a click or by its name.

10.5 A contract

```
(qss-define-contract my-libraries (-> (opt stringp) qss-table-p))
```

`(my-libraries 42)` is now reported as the fault of the caller. The contracts of the protocol check the methods above: had `qss-columns` returned something that is not a list of columns, the message would have named the method for `my-library`.

11 Reference

This chapter is written from the code by `doc/qss-manual-reference.el`.

11.1 Words

Word	Function	Meaning
`collect`	`qss-collect`	Return the rows of TABLE as a Lisp list, at the end of a pipe.
`columns`	`qss-columns-of`	Return the column names of TABLE as a table.
`count`	`qss-count`	Return the number of rows of TABLE.
`delete-rows`	`qss-delete-rows`	Delete the stored rows of TABLE, an SQLite query; return their number.
`each`	`qss-each`	Call FN on every row of TABLE; the results are the new rows.
`filter`	`qss-where`	Keep the rows of TABLE that satisfy COND.
`first`	`qss-first`	Return the first N rows of TABLE (default 1).
`get`	`qss-get`	Return the values of column COL of TABLE as a table with one column.
`group-by`	`qss-group-by`	Group the rows of TABLE by the columns in ARGS, with aggregates.
`into`	`qss-into`	Store TABLE in TARGET (a symbol) and return what was stored, as a source.
`join`	`qss-join`	Join TABLE with OTHER: rows whose join columns are equal, side by side.
`last`	`qss-last`	Return the last N rows of TABLE (default 1).
`nu`	`qss-nu-pipe`	Send TABLE through the Nushell COMMAND and return the result.
`reject`	`qss-reject`	Return TABLE with all columns except COLS (cut down as in `qss-select`).
`select`	`qss-select`	Return TABLE with only the columns COLS.
`sort-by`	`qss-sort-by`	Return TABLE sorted by column COL; DIR is :asc (default) or :desc.
`sql`	`qss-sql`	Return the query REL as text with its parameters.
`summarize`	`qss-summarize`	Return one row with the count of TABLE and the aggregates AGGS.
`to`	`qss-to`	Return TABLE as text in FORMAT, one of `qss-text-formats`.
`update-rows`	`qss-update-rows`	Set columns in the stored rows of TABLE, an SQLite query; return their number.
`where`	`qss-where`	Keep the rows of TABLE that satisfy COND.

11.2 Sources

Source	Function	Meaning
contracts	qss-contracts	Return a table of the functions with a contract and their contracts.
files	qss-files	Return a table of the files in DIR (default default-directory).
nu	qss-nu	Run the Nushell COMMAND and return its result as a table.
processes	qss-processes	Return a table of the running processes (from process-attributes).
run	qss-run	Run PROGRAM with ARGS, without a shell, and return its output as a table.
sqlite	qss-sqlite	Return the table TABLE of the SQLite file FILE as a query.
tables	qss-tables	Return the tables of the SQLite file FILE (default qss-database).

11.3 Options

All options are in the customization group qss (M-x customize-group RET qss).

Option	Default	Meaning
qss-contract-enabled	t	Non-nil means contracts are checked.
qss-copy-format	text	Format in which \w copies a table; see qss-to.
qss-data-directory	nil	Directory of the SQLite files of qss; nil means user-emacs-directory.
qss-database	nil	The central SQLite file of qss; nil means qss.sqlite3 in qss-data-directory.
qss-elisp-xref-database	nil	The SQLite file of the cross references.
qss-elisp-xref-directories	nil	Directories to index, searched recursively.
qss-elisp-xref-skip	"\\(?:-autoloads\\|-pkg\\|loaddefs\\|ldefs-boot\\|\\.dir-locals\\)\\.el"	Regexp of file names not to index (generated files).
qss-max-column-width	60	Maximum width of a text column without its own :width, in characters.
qss-menu-label-width	70	Maximum width of a label in the actions at point.
qss-nu-arguments	("--no-config-file")	Arguments for qss-nu-program before -c.
qss-nu-format	nuon	How data goes between Emacs and Nushell.
qss-nu-program	"nu"	The Nushell program.
qss-nu-size-columns	(size mem virtual total free used available)	Columns whose numbers are sizes in bytes, when qss-nu-format is json.
qss-repl-max-rows	30	Maximum number of rows a table shows inside the qss REPL.

Option	Default	Meaning
`qss-repl-prompt`	`"qss> "`	Prompt of the qss REPL.
`qss-sql-display-limit`	`1000`	Maximum number of rows a query fetches for display.

11.4 Faces

Face	Used for
`qss-accept`	Face of the presentations `qss-accept` offers.
`qss-edit-changed`	Face of a cell changed but not yet written.
`qss-edit-deleted`	Face of a row marked for deletion.
`qss-repl-directory`	Face of the directory in the qss REPL prompt.
`qss-repl-error`	Face of error messages in the qss REPL.
`qss-repl-prompt`	Face of the qss REPL prompt.
`qss-repl-title`	Face of the title line above a table in the qss REPL.
`qss-table`	Face of qss tables.

11.5 Presentation types

"Text" says whether typed text can be read as the type.

Type	Below	Text	Meaning
`db-table`	`object`		A table of an SQLite file.
`directory`	`file`	yes	A directory.
`duration`	`integer`	yes	A duration in nanoseconds; "3hr" can be typed.
`file`	`object`	yes	A file or directory.
`integer`	`number`	yes	A whole number.
`number`	`value`	yes	A number.
`object`	`t`		Something with actions: a row object.
`process`	`object`	yes	A system process.
`row`	`object`		A row of data (a property list).
`size`	`integer`	yes	A size in bytes; "10kb" can be typed.
`string`	`value`	yes	Text.
`symbol`	`value`	yes	A symbol.
`time`	`value`	yes	A point in time; "2026-10-22" or "2026-10-22 07:30" can be typed.
`value`	`t`	yes	A plain value: number, text, time.

11.6 Commands with typed arguments

Command	In the menu	Arguments
`qss-compare-files`	Compare with file	`a file`, `b file`
`qss-copy-file-into`	Copy into directory	`file file`, `dir directory`

Command	In the menu	Arguments
`qss-files-larger-than`	Files larger than this, here	`size size`

11.7 Named contracts

Named contract	Contract and meaning
`qss-action`	`(cons/c stringp functionp)` An action: (LABEL . FUNCTION).
`qss-aggregate`	`(and/c consp (lambda (a) (memq (car a) qss-aggregates)))` An aggregate: (FN COLUMN) or (FN COLUMN :as NAME), FN one of sum avg min max count.
`qss-column-name`	`(or/c symbolp stringp)` A column name: name, :name or "name".
`qss-column-spec`	`(or/c symbolp (cons/c symbolp qss-interface--column-plist-p))` A column: a field name, or (NAME . PLIST) with :type, :format, :align, :width.
`qss-format`	`(or/c (one-of text org md csv tsv) stringp)` A text format of `qss-to`.
`qss-presentation-type-name`	`qss-presentation-type-p` A presentation type: file, directory, size, value ... or t.
`qss-result`	`(or/c qss-table-p qss-relation-p)` What most words return: a table, or a query on SQLite.
`qss-tabular`	`(or/c qss-table-p qss-relation-p list)` What the words take: a table, a query, or a list of rows.

11.8 Contracts

The 51 functions of the interface with their contracts, as `qss-interface.el` defines them.

Function	Contract
`qss-accept`	`(-> qss-presentation-type-name (opt stringp) any)`
`qss-actions`	`(-> any (listof qss-action))`
generic	
`qss-actions-at-point`	`(-> (listof (listof qss-action)))`
`qss-as-input`	`(-> any qss-result)`
generic	
`qss-as-table`	`(-> any qss-table-p)`
generic	
`qss-collect`	`(-> qss-tabular list)`
generic	
`qss-columns`	`(-> any (listof qss-column-spec))`
generic	
`qss-columns-of`	`(-> qss-tabular qss-table-p)`
generic	
`qss-commands-for`	`(-> any qss-presentation-type-name (listof qss-action))`
`qss-continue`	`(-> qss-tabular consp any)`

Function	Contract
qss-copy-table-as	(-> qss-tabular qss-format any)
qss-count	(-> qss-tabular natnump)
generic	
qss-define-presentation-type	(-> symbolp qss-presentation-type-name (opt functionp) (opt stringp) symbolp)
qss-delete-rows	(-> qss-tabular (rest any) natnump)
generic	
qss-demo-database	(-> stringp)
qss-each	(-> qss-tabular functionp qss-result)
generic	
qss-file-name	(-> any stringp)
generic	
qss-files	(-> (opt stringp) qss-table-p)
qss-first	(-> qss-tabular (opt natnump) qss-result)
generic	
qss-get	(-> qss-tabular qss-column-name qss-result)
generic	
qss-group-by	(-> qss-tabular (rest (or/c qss-column-name qss-aggregate))
generic	qss-result)
qss-into	(-> qss-tabular symbolp (rest any) any)
generic	
qss-join	(-> qss-tabular qss-tabular (rest any) qss-result)
generic	
qss-last	(-> qss-tabular (opt natnump) qss-result)
generic	
qss-make-table	(-> stringp list (opt (listof qss-column-spec)) (opt functionp) qss-table-p)
qss-nu	(-> stringp any)
qss-nu-pipe	(-> qss-tabular stringp any)
qss-nuon-read	(-> stringp any)
qss-present	(-> any (opt qss-presentation-type-name) (opt stringp) any)
qss-presentation-label	(-> any stringp)
generic	
qss-presentation-parse	(-> qss-presentation-type-name stringp any)
qss-presentation-subtype-p	(-> qss-presentation-type-name qss-presentation-type-name any)
qss-presentation-type	(-> any qss-presentation-type-name)
generic	
qss-processes	(-> qss-table-p)
qss-reject	(-> qss-tabular (rest qss-column-name) qss-result)
generic	
qss-run	(-> stringp (rest stringp) qss-table-p)
qss-select	(-> qss-tabular (rest qss-column-name) qss-result)
generic	
qss-show	(-> qss-table-p (opt any) bufferp)
qss-sort-by	(-> qss-tabular qss-column-name (opt symbolp) qss-result)
generic	
qss-sql	(-> any stringp)
qss-sql-name-key	(-> stringp symbolp)
qss-sql-rowid	(-> any (or/c null integerp))
qss-sql-schema	(-> stringp (listof (cons/c stringp list)))

Function	Contract
qss-sql-write	(-> qss-table-p stringp stringp (opt (one-of replace append)) natnump)
qss-sqlite	(-> stringp (opt (or/c stringp symbolp)) qss-relation-p)
qss-stored-fields	(-> any (listof symbolp))
generic	
qss-summarize	(-> qss-tabular (rest qss-aggregate) qss-result)
generic	
qss-tables	(-> (opt stringp) qss-table-p)
qss-to	(-> qss-tabular qss-format stringp)
generic	
qss-update-rows	(-> qss-tabular (rest any) natnump)
generic	
qss-where	(-> qss-tabular (or/c qss-condition-p functionp) qss-result)
generic	

12 Where the ideas come from

Little in `qss` is new; it puts old ideas together in Emacs.

Nushell Data as tables, not text; one vocabulary for every source; the pipe. The names of the words, the units `10kb` and `3hr`, and NUON come from there. `https://www.nushell.sh/`

dbplyr A pipe over a database table stays a query and becomes SQL only when the rows are needed; the same words work in memory and in the database. `https://dbplyr.tidyverse.org/`

SQL The meaning of the comparisons: `like`, `in`, and NULL as unknown in a three-valued logic.

SQLite One file as the store of everything, source and target alike.

CLIM and Dynamic Windows Presentations, presentation types, `accept`, commands with typed arguments offered on the objects shown. McCLIM is a free implementation: `https://mcclim.common-lisp.dev/`.

CLOS Generic functions with methods per type (`cl-generic` in Emacs): the protocol of rows, so a file stays a file and own types fit in with a few methods.

Racket contracts Contracts at the boundaries, and blame: the caller or the function. R. B. Findler, M. Felleisen, *Contracts for Higher-Order Functions*, ICFP 2002. `https://docs.racket-lang.org/guide/contracts.html`

Emacs `vtable` for the tables, the built-in `sqlite`, advice for the contracts, text properties for the presentations, IELM for the shape of the REPL — and the conviction that the editor is the place to work in.

A Files and tests

`qss.el`	package headers, loads everything, the commands to run the tests
`qss-table.el`	tables, display, the protocol of rows, `files`, `processes`, `run`
`qss-pipe.el`	the words, conditions, `qss->`, `to`, `into`, groups, joins
`qss-sql.el`	SQLite as a source and a store; joins and groups as SQL
`qss-nu.el`, `qss-nuon.el`	Nushell; NUON reader and writer
`qss-present.el`	presentation types, `qss-present`, `qss-accept`, commands
`qss-menu.el`	actions at point, `a`, the right mouse button, `w`
`qss-edit.el`	changing rows of SQLite tables; `update-rows`, `delete-rows`
`qss-repl.el`	`M-x qss`
`qss-contract.el`	contracts with blame (stands alone)
`qss-interface.el`	the interface of `qss` with its contracts
`qss-examples.el`	`M-x qss-examples`
`doc/`	this manual, its example and the writer of its reference

`M-x qss-run-tests` runs the tests in Emacs; from a shell:

```
emacs --batch -L /path/to/qss -l qss -f qss-run-tests-batch
```

The tests run with the contracts on, so every test checks the code against the contracts as well. The SQLite tests are skipped without SQLite, the Nushell tests without `nu`.